

DataDSL — Lexer Ad-hoc (Token.hs) Explicado

David Souza

Documentação técnica do primeiro analisador léxico do projeto DataDSL, implementado manualmente para consolidar os conceitos de análise léxica estudados na disciplina de Construção de Compiladores.

1 Contexto e objetivo

O DataDSL é uma linguagem declarativa para pipelines simples de transformação de dados, compilada para código Python/pandas. Antes de utilizar o Alex (gerador de analisadores léxicos), foi implementado um lexer **ad-hoc** — ou seja, escrito manualmente, caractere por caractere — cobrindo palavras-chave, identificadores e números. O objetivo dessa etapa foi fixar, na prática, os mecanismos de análise léxica vistos em sala: acumulação de caracteres, o critério do maior prefixo (*maximal munch*) e a distinção entre palavras reservadas e identificadores.

A versão final do projeto utiliza o Alex para gerar o lexer completo (incluindo operadores e strings), mas esta implementação manual permanece documentada por seu valor didático e como registro do processo de aprendizado.

2 Tipos de dados

2.1 Line e Column

```
type Line = Int
type Column = Int
```

Apelidos de tipo para `Int`, usados para tornar as assinaturas de função mais legíveis, indicando que aquele inteiro representa uma linha ou coluna do código-fonte.

2.2 Token

```
data Token = Token
  { pos :: (Line, Column)
  , lexeme :: Lexeme
  } deriving (Show)
```

Representa um token completo: a posição onde ele aparece no código-fonte (linha e coluna) e a sua categoria (o `Lexeme`). Guardar a posição é essencial para mensagens de erro precisas mais adiante, no parser.

2.3 Lexeme

```
data Lexeme
  = TLoad
```

```

| TFilter
| TSelect
| TIdent String
| TNumber Int
deriving (Show)

```

Enumera as categorias de token reconhecidas por esta versão do lexer. Repare que `TLoad`, `TFilter` e `TSelect` não carregam nenhum valor — o simples fato de existirem já identifica o token, sem informação adicional. Já `TIdent` e `TNumber` carregam um valor (o nome da coluna ou o número lido), porque essa informação é necessária para as fases seguintes do compilador.

2.4 State

```

type State = (Line, Column, String, [Token])

```

O estado interno do lexer enquanto ele percorre a entrada, caractere por caractere: a linha e coluna atuais, uma `String` usada como acumulador (para juntar dígitos formando um número, ou letras formando uma palavra) e a lista de tokens já reconhecidos.

3 Funções

3.1 lexer — função principal

```

lexer :: String -> Either String [Token]
lexer = finish . foldl step base
  where
    base = Right (1, 1, "", [])
    step (Left e) _ = Left e
    step (Right s) c = transition s c
    finish = either Left (Right . extract)
    extract (l, c, s, ts)
      | null s = reverse ts
      | all isAlpha s = let t = Token (l, c - length s)
                           (classificarPalavra (reverse s))
                        in reverse (t : ts)
      | all isDigit s = let n = read (reverse s)
                        in let t = Token (l, c - length s) (TNumber n)
                           in reverse (t : ts)

```

Ponto de entrada do módulo. Utiliza `foldl` para percorrer a string de entrada inteira, aplicando a função `step` a cada caractere e acumulando o `State` resultante. Se algum caractere causar erro (`Left`), os caracteres seguintes deixam de ser processados e o erro é propagado até o final. Ao terminar, a função `extract` trata o caso em que ainda resta algo acumulado (por exemplo, quando a entrada termina sem um espaço após o último identificador ou número), gerando o último token pendente antes de devolver a lista final — já na ordem correta, com `reverse`.

3.2 mkDigits

```

mkDigits :: State -> Char -> Either String State
mkDigits state@(l, col, s, ts) c
  | null s = ...
  | all isDigit s = ...
  | otherwise = Left $ unexpectedCharError l col c

```

Finaliza uma sequência de dígitos acumulada, transformando-a em um token `TNumber`. Se nada havia sido acumulado, apenas atualiza a posição (linha/coluna) sem gerar token.

3.3 mkWord

Estruturalmente idêntica a `mkDigits`, mas finaliza uma sequência de letras acumulada. Em vez de criar diretamente um `TNumber`, delega a decisão à função `classificarPalavra`, que decide se a palavra reconhecida é uma palavra-chave ou um identificador comum.

3.4 classificarPalavra

```
classificarPalavra :: String -> Lexeme
classificarPalavra s
  | s == "load" = TLoad
  | s == "filter" = TFilter
  | s == "select" = TSelect
  | otherwise = TIdent s
```

Decide se uma palavra já reconhecida é uma palavra reservada da linguagem ou um identificador. Esta é a solução adotada para o problema clássico de distinguir palavras-chave de identificadores (por exemplo, *if* vs. *ift*): em vez de o autômato léxico distinguir os dois casos diretamente, uma única regra reconhece qualquer sequência de letras, e esta função separada consulta uma pequena tabela de palavras conhecidas após o reconhecimento léxico.

3.5 transition

```
transition :: State -> Char -> Either String State
transition state@(line, col, t, ts) c
  | c == '\n' && null t = mkDigits state c
  | c == '\n' && all isDigit t = mkDigits state c
  | c == '\n' && all isAlpha t = mkWord state c
  | isSpace c && null t = mkDigits state c
  | isSpace c && all isDigit t = mkDigits state c
  | isSpace c && all isAlpha t = mkWord state c
  | isDigit c = Right (line, col + 1, c : t, ts)
  | isAlpha c = Right (line, col + 1, c : t, ts)
  | otherwise = Left $ unexpectedCharError line col c
```

Função central do lexer: para cada caractere lido, decide o que fazer. Um ponto de destaque é a decisão sobre **qual** função de finalização chamar (`mkDigits` ou `mkWord`) quando surge um espaço ou quebra de linha: essa decisão é feita observando o conteúdo já acumulado em `t` — se são apenas dígitos ou apenas letras.

Limitação conhecida: essa estratégia (decidir pelo conteúdo já acumulado) funciona bem para números e identificadores, que nunca misturam letras e dígitos nesta gramática, mas não se generaliza diretamente para strings (que podem conter qualquer caractere entre aspas). Essa é uma das razões pelas quais a versão final do lexer foi migrada para o Alex.

3.6 unexpectedCharError

```
unexpectedCharError :: Line -> Column -> Char -> String
unexpectedCharError l c char =
  "Caractere inesperado '" ++ [char] ++ "' na linha "
  ++ show l ++ ", coluna " ++ show c
```

Constrói uma mensagem de erro léxico legível, informando exatamente qual caractere causou o problema e em que posição do código-fonte ele apareceu.

4 Limitações desta versão e próximos passos

Este lexer ad-hoc reconhece apenas palavras-chave (`load`, `filter`, `select`), identificadores e números inteiros. Ficam pendentes:

- Operadores de comparação (`>`, `<`, `>=`, `<=`, `==`), que exigem *look-ahead* de um caractere para decidir entre um operador simples e um composto.
- Vírgula, como separador de colunas em `select`.
- Strings delimitadas por aspas, que exigem um mecanismo de estado diferente do usado para números/identificadores, pois podem conter qualquer caractere entre as aspas.

Esses três pontos foram resolvidos na versão final do lexer, gerada automaticamente a partir de uma especificação de expressões regulares usando o Alex — aplicando diretamente a teoria de autômatos finitos (construção de Thompson e *subset construction*) estudada na disciplina.